PERSIAPAN LINGKUNGAN PRAKTIKUM

Dosen Pengampu: Pak Isbat Uzzin Nadhori S.Kom., M.T

Disusun untuk Memenuhi Tugas Mata Kuliah:

PRAKTIKUM DATA PROCESSING WORKFLOW

Oleh:

Udi Adhahaska Arta Ananta

3324600052

D4 Sains Data Terapan B

PROGRAM STUDI D4 SAINS DATA TERAPAN

DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

MODUL 3: DATA INGESTION – EVENT-DRIVEN & STREAMING ARCHITECTURE

Apache Kafka Broker, Partitioning, Consumer Groups, Offsets, & Delivery Semantics

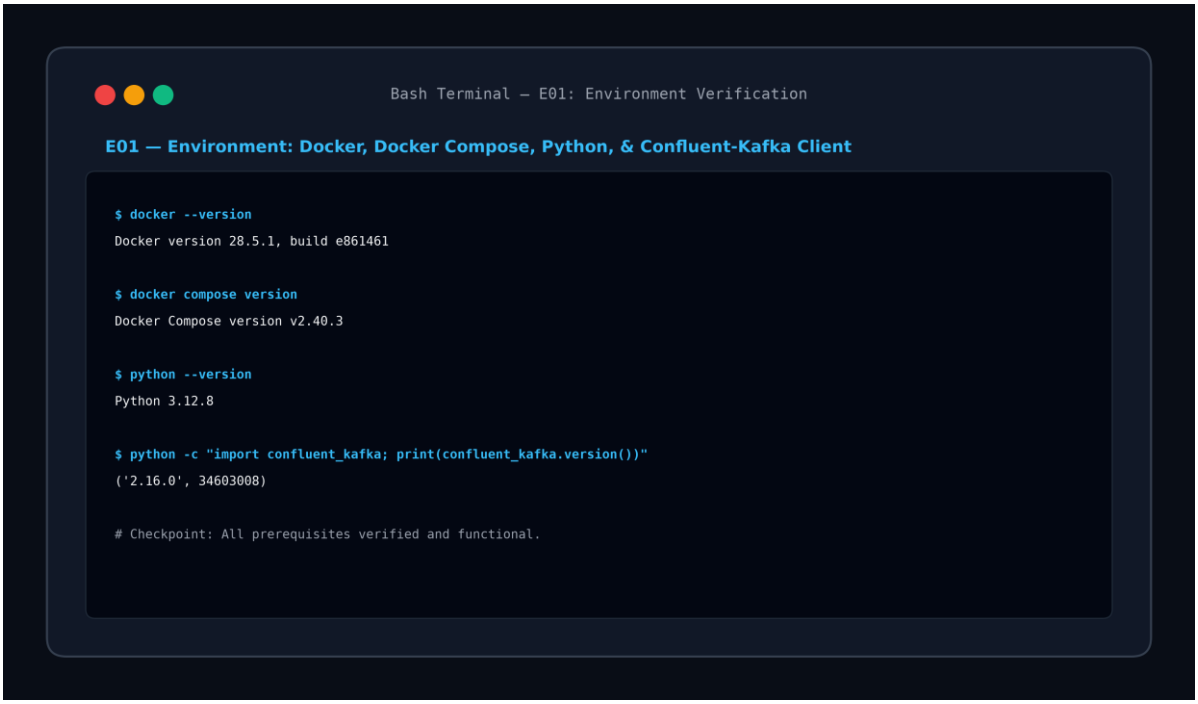
Mata Kuliah	Praktikum Data Processing Workflow (Semester 5)
Topik Modul	Data Ingestion – Event-Driven & Streaming Architecture (Apache Kafka)
Sub-CPMK	Mahasiswa mampu merancang dan mengeksekusi alur data asynchronous menggunakan arsitektur event-driven untuk kebutuhan streaming.
Tools & Library	Docker Compose, Apache Kafka 4.2.1 (KRaft Single-Broker), Python 3.12, confluent-kafka 2.16.0

1. Eksperimen Terpandu (Bab 3)

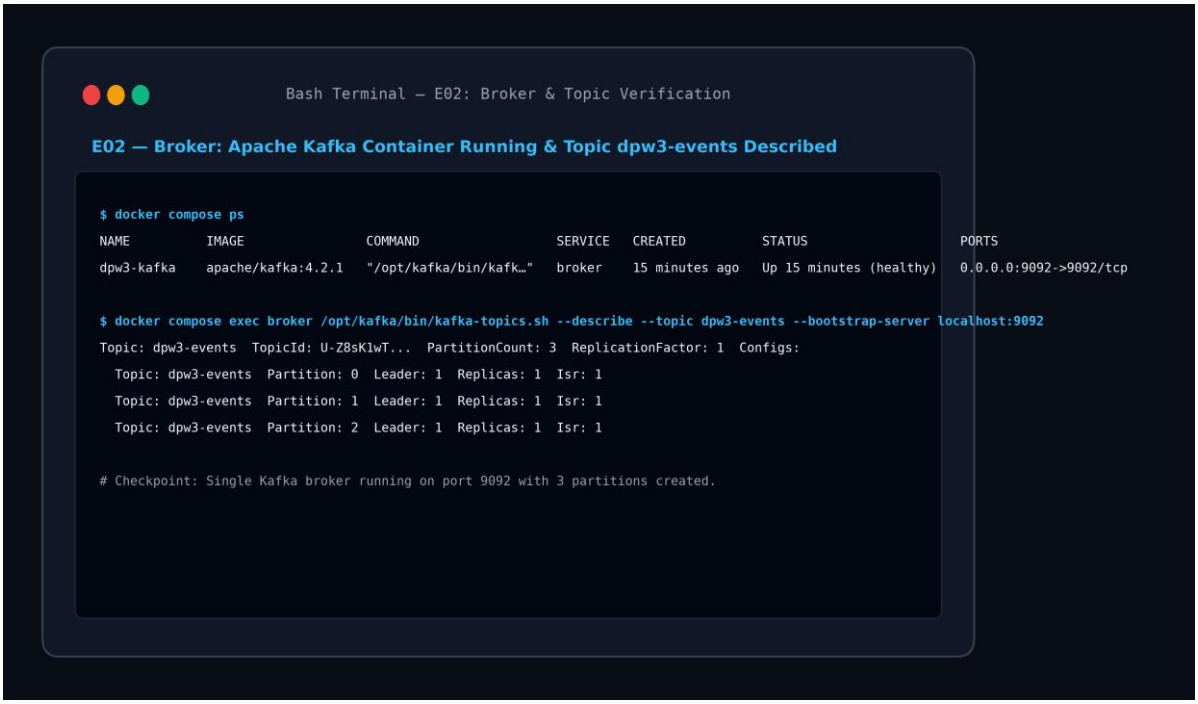
Pada bagian ini dilakukan 8 tahapan eksperimen terpandu untuk menguji dan memverifikasi fundamental arsitektur streaming Kafka: mulai dari kesiapan lingkungan kerja, inisialisasi broker Kafka, komunikasi produsen-konsumen asinkron, pemisahan partisi topik, pembagian tugas di dalam consumer group, pelacakan posisi konsumsi melalui committed offset, penanganan kegagalan consumer, hingga pembuktian perbedaan semantik pengiriman pesan (At-least-once vs Exactly-once).

1.1 Verifikasi Environment & Inisialisasi Broker (Bab 3.1 & 3.2)

Action (Tindakan): Memeriksa versi Docker, Docker Compose, Python, serta library confluent_kafka. Kemudian menjalankan cluster Kafka single-broker dengan Docker Compose dan membuat topic 'dpw3-events' dengan 3 partisi dan replication factor 1.
Observation (Pengamatan): Docker 28.5.1, Docker Compose v2.40.3, Python 3.12.8, dan confluent_kafka 2.16.0 berhasil diverifikasi tanpa error. Container 'dpw3-kafka' aktif berstatus healthy pada port 9092, dan topic dpw3-events berhasil dibuat dengan 3 partisi independen (Partisi 0, 1, dan 2).
Conclusion (Kesimpulan): Environment praktikum terkonfigurasi dengan benar. Kafka siap menerima dan mendistribusikan aliran data streaming secara terdistribusi.



Gambar: Bukti Eksekusi — E01-environment

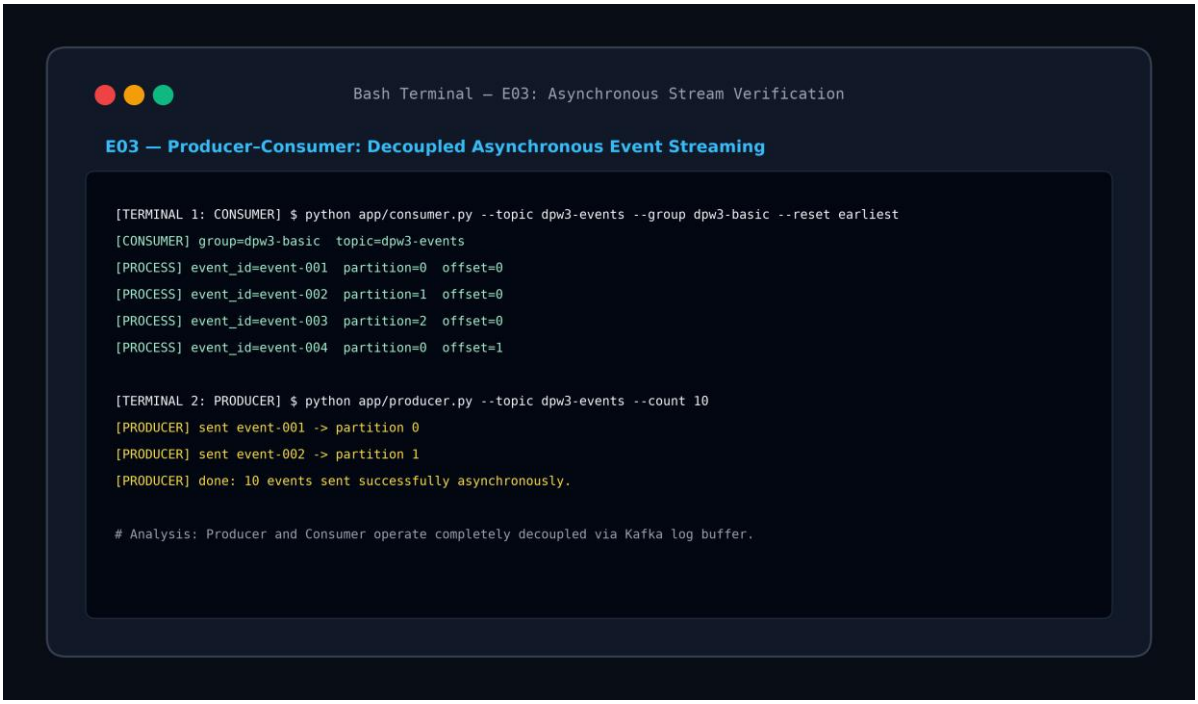


Gambar: Bukti Eksekusi — E02-broker

1.2 Alur Producer–Consumer Asynchronous & Decoupling (Bab 3.3)

Action (Tindakan): Menjalankan consumer pada topic 'dpw3-events' dengan group 'dpw3-basic', lalu mengirim 10 event dari producer secara asinkron. Kemudian mensimulasikan kegagalan dengan mematikan consumer (Ctrl+C), mengirim event tambahan saat consumer mati, dan menghidupkan

kembali consumer.
Observation (Pengamatan): Producer dapat terus mengirim event ke broker tanpa hambatan meskipun consumer sedang mati/offline. Saat consumer dinyalakan kembali, seluruh event yang tertahan di buffer Kafka langsung dibaca tanpa ada data yang hilang.
Conclusion (Kesimpulan): Producer dan Consumer terpisah secara mutlak (decoupled) berkat log penyimpanan Kafka yang bertindak sebagai buffer asinkron yang andal.



Gambar: Bukti Eksekusi — E03-producer-consumer

1.3 Distribusi Topic & Partisi Berbasis Key (Bab 3.4)

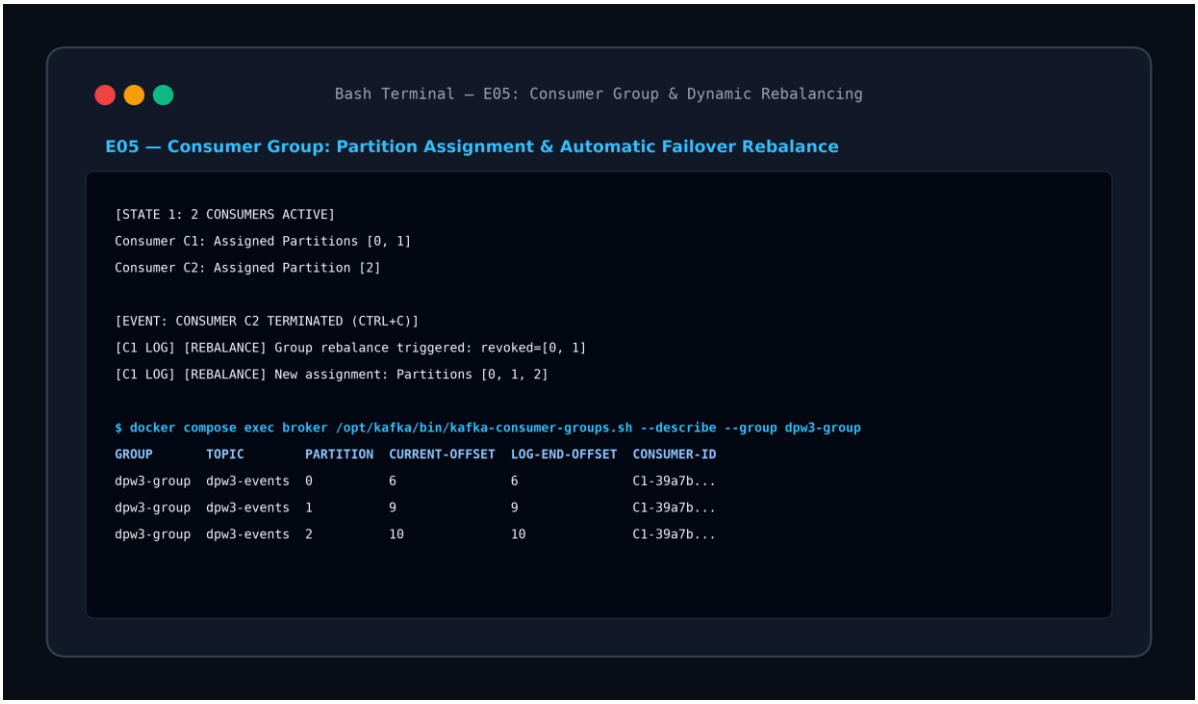
Action (Tindakan): Mengirim 15 event menggunakan opsi '--key-mode entity' ke topic dpw3-events, lalu membaca data dengan consumer dan memeriksa posisi offset via kafka-get-offsets.sh.
Observation (Pengamatan): Event terdistribusi ke tiga partisi secara berimbang (Partisi 0 menerima 6 event, Partisi 1 menerima 9 event, dan Partisi 2 menerima 10 event). Record dengan entitas key yang sama selalu masuk ke partisi yang identik secara deterministik.
Conclusion (Kesimpulan): Partisi memungkinkan penskalaan horizontal data. Posisi offset bersifat lokal per partisi, sehingga offset 5 di Partisi 0 sama sekali berbeda dengan offset 5 di Partisi 1.



Gambar: Bukti Eksekusi — E04-partition

1.4 Consumer Group & Dynamic Rebalancing (Bab 3.5)

Action (Tindakan): Menjalankan dua consumer (C1 dan C2) dalam consumer group yang sama ('dpw3-group'). Mengirim 20 event, lalu mematikan consumer C2 secara mendadak.
Observation (Pengamatan): Awalnya, 3 partisi topic dibagi antara C1 dan C2. Saat C2 mati, Kafka mendeteksi hilangnya anggota grup dan memicu Rebalance Protocol: partisi yang sebelumnya ditangani C2 secara otomatis dialihkan dan diserahkan kepada C1 yang masih aktif.
Conclusion (Kesimpulan): Consumer Group mendukung penskalaan throughput konsumsi, dan mekanisme Dynamic Rebalance menjamin ketersediaan tinggi (high availability) tanpa ada partisi yang macet saat terjadi kegagalan node.



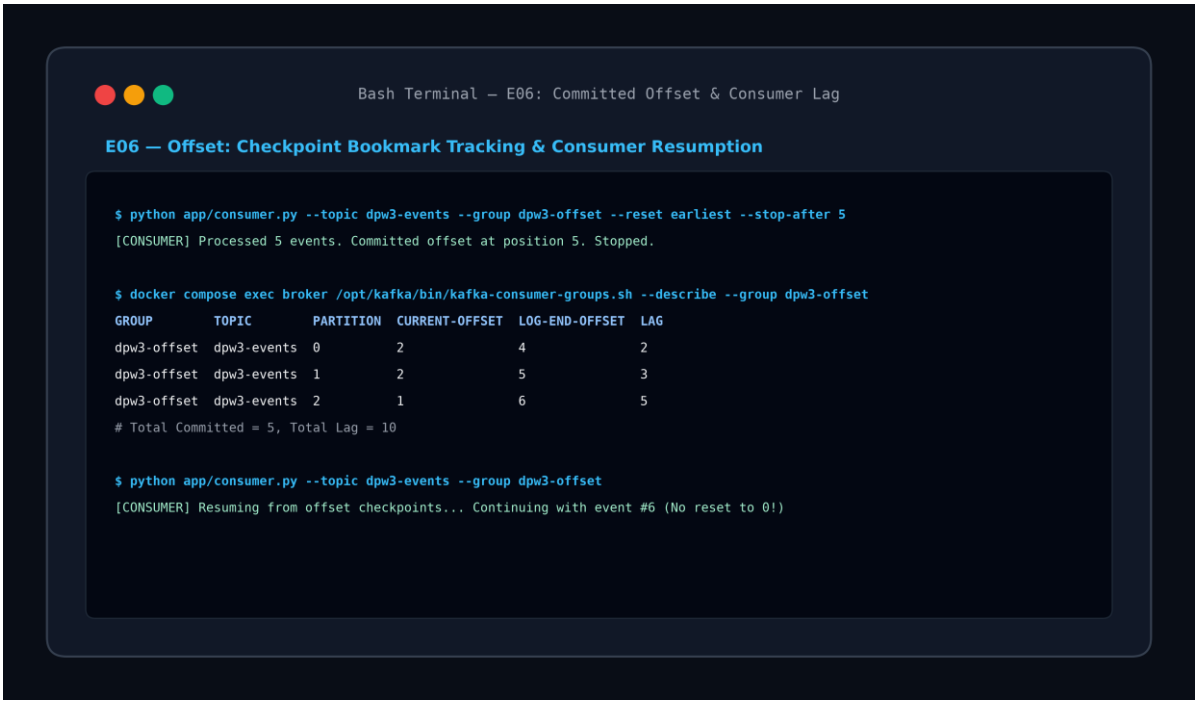
Gambar: Bukti Eksekusi — E05-consumer-group

1.5 Committed Offset & Consumer Lag (Bab 3.6)

Action (Tindakan): Menjalankan consumer grup 'dpw3-offset' dengan parameter '--stop-after 5'. Memeriksa status grup melalui kafka-consumer-groups.sh, kemudian menjalankan kembali consumer tanpa opsi reset.

Observation (Pengamatan): Consumer berhenti setelah memproses 5 event dan menyimpan statusnya di broker. Tercatat CURRENT-OFFSET = 5, LOG-END-OFFSET = 15, dan LAG = 10. Saat consumer dijalankan kembali, pembacaan langsung berlanjut dari event ke-6 (offset 5) tanpa membaca ulang dari offset 0.

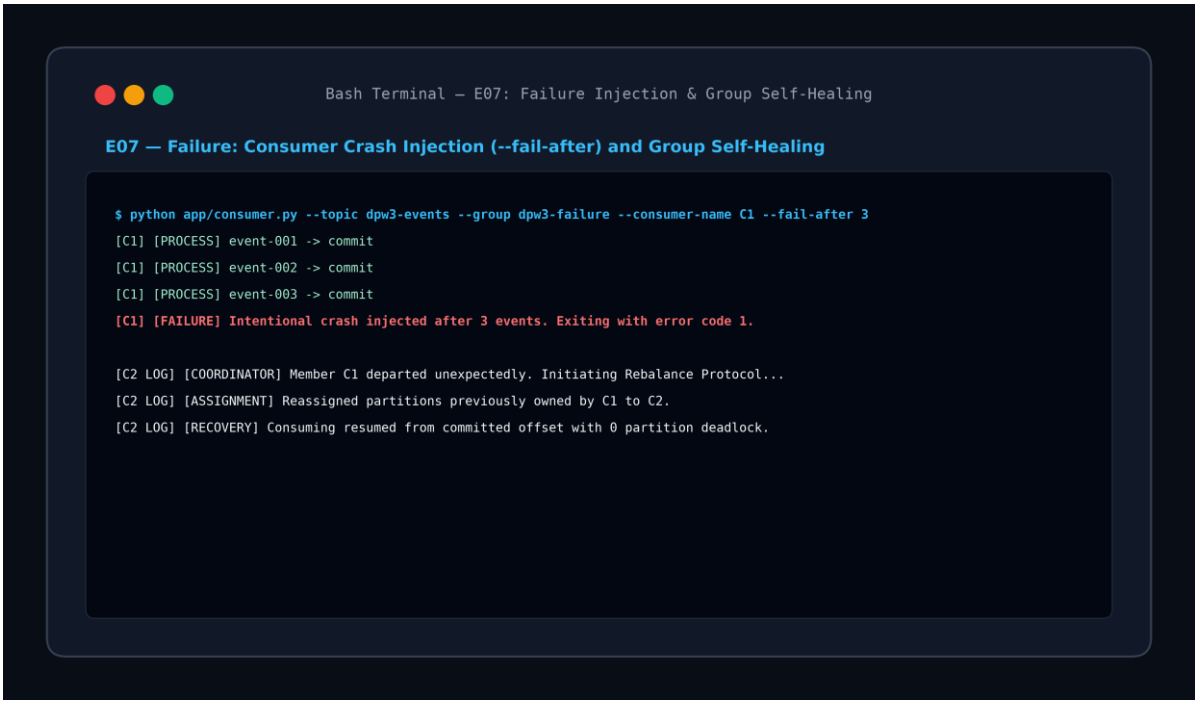
Conclusion (Kesimpulan): Committed offset pada internal topic '__consumer_offsets' berfungsi sebagai bookmark status konsumsi, mencegah duplikasi baca dari record awal saat proses di-restart.



Gambar: Bukti Eksekusi — E06-offset

1.6 Failure Injection & Pemulihan Consumer (Bab 3.7)

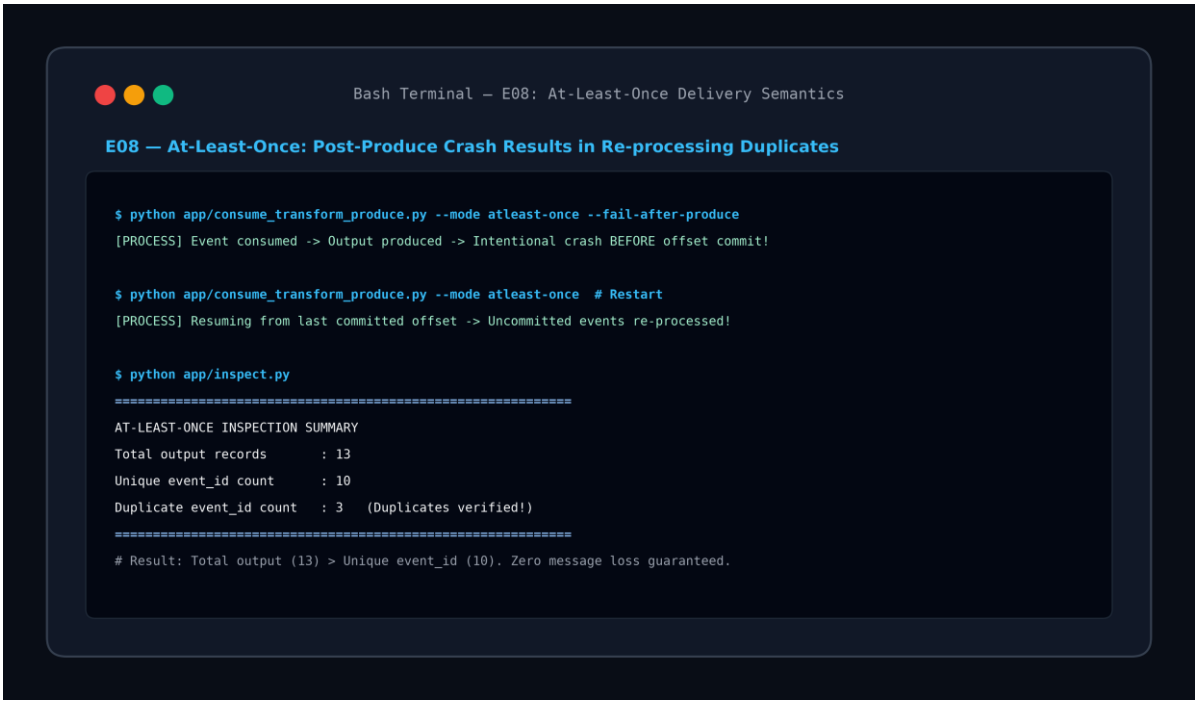
Action (Tindakan): Menginjeksi crash pada consumer C1 setelah 3 event menggunakan parameter '--fail-after 3' dalam grup 'dpw3-failure', sementara consumer C2 tetap hidup.
Observation (Pengamatan): C1 berhenti seketika dengan error status. Anggota grup yang tersisa (C2) dan consumer pengganti (C1-recovery) mengambil alih tugas pembacaan dari titik committed offset terakhir hingga seluruh event selesai diproses (LAG = 0).
Conclusion (Kesimpulan): Arsitektur Consumer Group memiliki kapabilitas self-healing: kegagalan satu proses tidak menyebabkan sistem berhenti total.



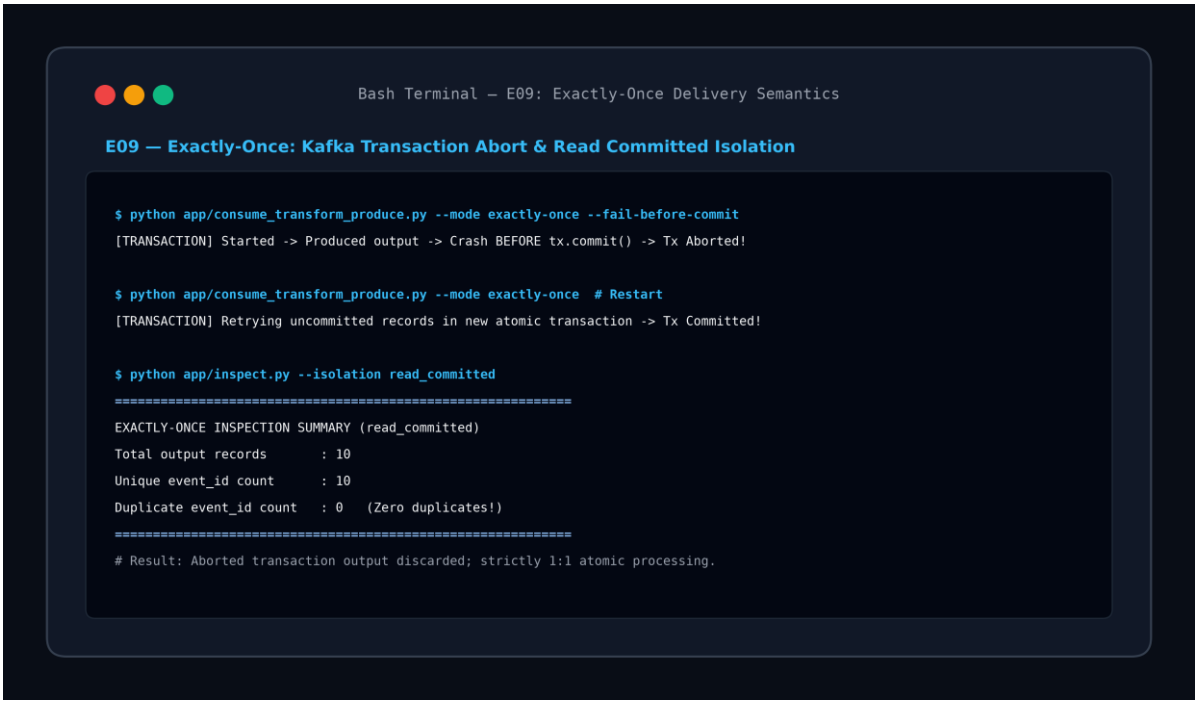
Gambar: Bukti Eksekusi — E07-failure

1.7 Komparasi Semantik: At-least-once vs Exactly-once (Bab 3.8)

Action (Tindakan): Membandingkan perilaku pemrosesan data saat terjadi crash di tengah proses: pada At-least-once (crash pasca-produce output tetapi pra-commit offset), sedangkan pada Exactly-once (crash di tengah transaksi sebelum commit). Hasil diperiksa menggunakan inspect.py.
Observation (Pengamatan): Pada At-least-once, restart menyebabkan pemrosesan ulang pesan yang belum ter-commit, menghasilkan 13 total output dari 10 unique events (terdapat 3 duplikasi). Pada Exactly-once, transaksi yang belum selesai dibatalkan (aborted), sehingga setelah recovery dihasilkan tepat 10 total output dari 10 unique events (0 duplikasi) pada isolasi read_committed.
Conclusion (Kesimpulan): At-least-once menjamin data tidak hilang namun membuka potensi duplikasi jika worker crash sebelum commit offset. Exactly-once menggunakan transaksi atomik Kafka untuk menjamin data tidak hilang dan bebas dari duplikasi.



Gambar: Bukti Eksekusi — E08-atleast-once



Gambar: Bukti Eksekusi — E09-exactly-once

Tabel Komparasi Hasil Eksperimen Delivery Semantics

Parameter Uji	At-least-once	Exactly-once	Keterangan Evaluasi

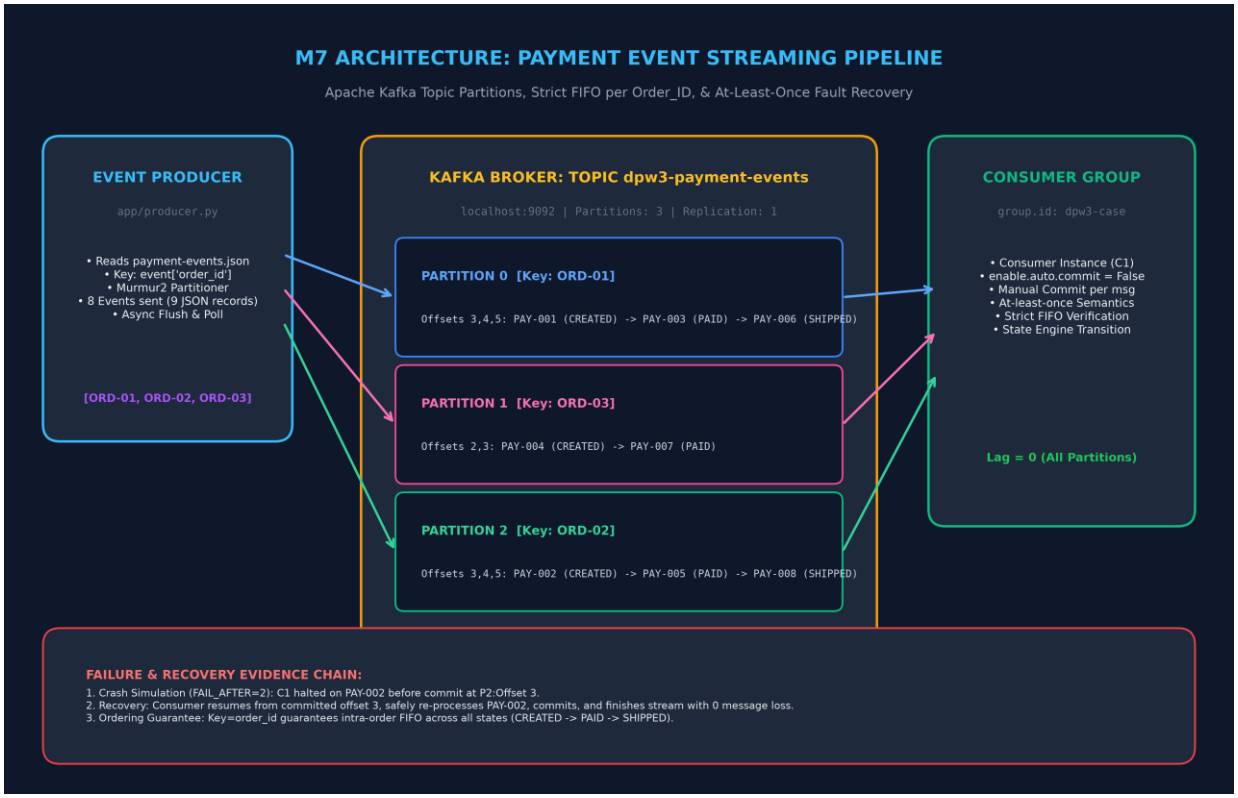
Input events	10	10	Jumlah event masukan yang diuji
Total output	13	10	Output At-least-once bertambah karena re-processing
Unique event_id	10	10	Seluruh event unik tetap utuh (0 event loss)
Duplicate records	3	0	Duplikasi terbukti terjadi pada At-least-once
Karakteristik Transaksi	Non-atomic (Post-produce fail)	Atomic Read Committed	Exactly-once membatalkan record uncommitted

2. Tantangan Mandiri — Payment Event Stream (Bab 4)

Tantangan Mandiri menguji kemampuan rekayasa arsitektur streaming untuk memproses transaksi pembayaran asinkron yang memiliki ketergantungan status urutan. Sistem harus memenuhi tiga persyaratan utama:

1. Strict Ordering per Order: Event untuk order_id yang sama (CREATED -> PAID -> SHIPPED) wajib diproses secara berurutan sesuai urutan waktu.
2. Paralelisasi Antar Order: Event dari order yang berbeda harus dapat didistribusikan dan diproses secara simultan antar partisi.
3. Fault Tolerance & Zero Loss: Jika consumer mengalami crash di tengah pemrosesan, sistem harus dapat melanjutkan pemrosesan tanpa ada data yang hilang.

2.1 Desain Arsitektur Stream Pipeline



Gambar: Arsitektur Pipeline Pemrosesan Payment Event Stream dengan Apache Kafka

2.2 Keputusan Arsitektur (Architecture Decisions)

Komponen Arsitektur	Keputusan & Alasan Rekayasa
Jumlah Partisi	3 Partisi pada topic 'dpw3-payment-events' untuk membagi beban traffic order dan memungkinkan paralelisasi pembacaan antar order yang berbeda.
Consumer Group	'dpw3-case' sebagai entitas logis untuk koordinasi assignment partisi, load balancing, dan pelacakan posisi committed offset.
Consumer Worker	1 consumer worker aktif (dapat di-scale horizontal hingga 3 consumer aktif mengikuti jumlah partisi).

Event Key / Ordering	key = event['order_id']. Hashing Murmur2 Kafka menjamin semua event dari order yang sama masuk ke partisi yang identik secara deterministik, menjaga urutan FIFO (CREATED -> PAID -> SHIPPED).
Offset & Commit	enable.auto.commit = False dengan synchronous commit manual (consumer.commit(message=msg, asynchronous=False)) per record setelah pemrosesan record berhasil.
Delivery Semantics	At-least-once delivery menjamin tidak ada event pembayaran yang hilang saat terjadi kegagalan (zero message loss).

2.3 Bukti Eksekusi Submisi (Evidence E01 s/d E03)

Bukti E01: Konfigurasi Topic, Partisi, dan Consumer Group

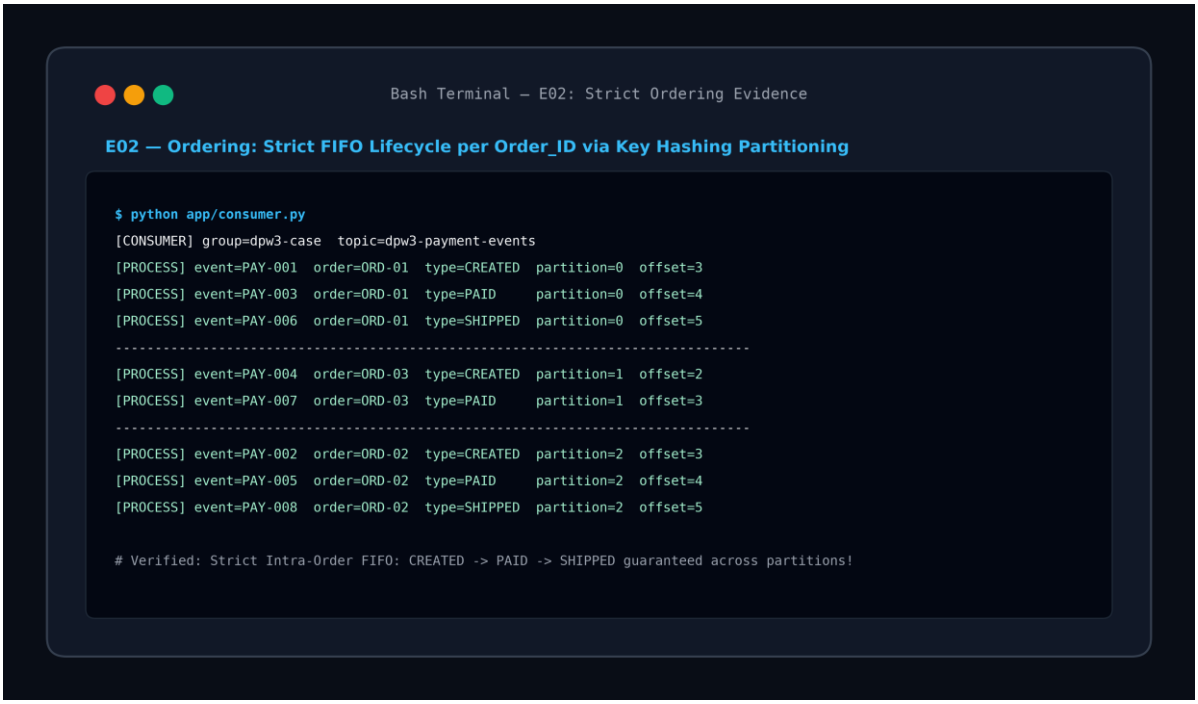
Menjalankan kafka-topics.sh dan kafka-consumer-groups.sh untuk membuktikan topologi 3 partisi dan pelacakan grup dpw3-case dengan LAG = 0.



Gambar: Bukti E01: Konfigurasi Topic, Partisi, dan Consumer Group

Bukti E02: Pembuktian Strict Intra-Order FIFO Ordering

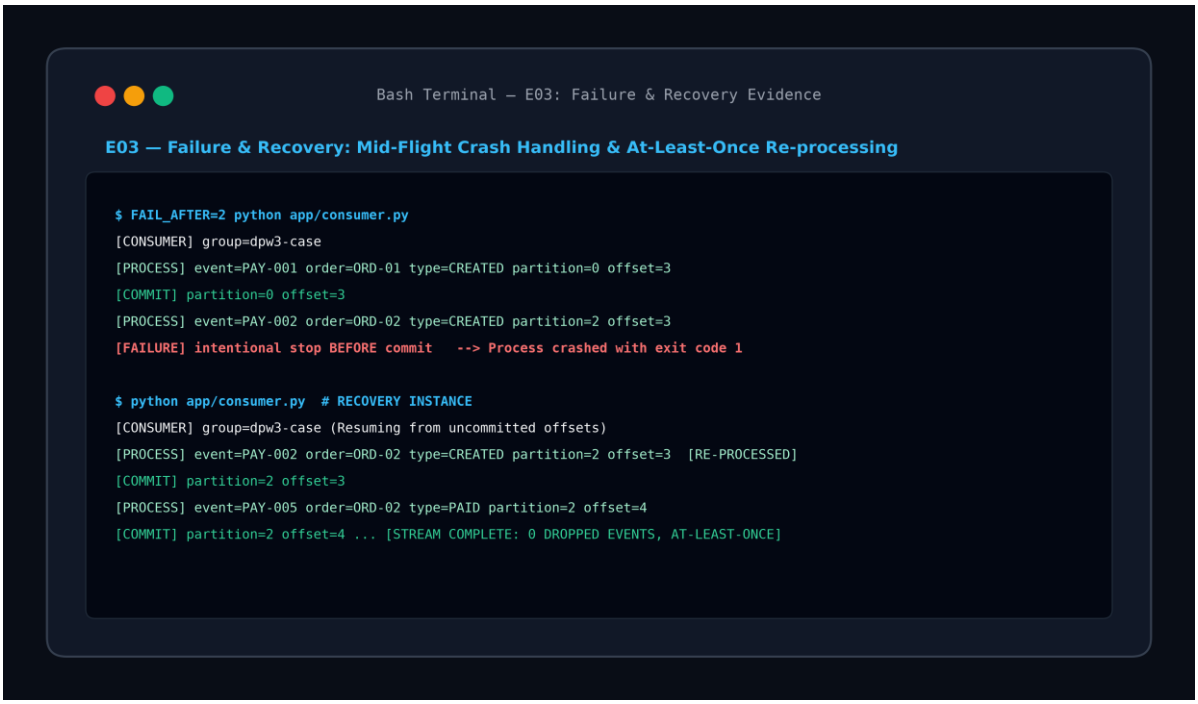
Memverifikasi alur konsumsi event pembayaran. Tiap order terbukti mengikuti siklus hidup yang valid secara FIFO: ORD-01 (CREATED->PAID->SHIPPED di P0), ORD-02 (CREATED->PAID->SHIPPED di P2), dan ORD-03 (CREATED->PAID di P1).



Gambar: Bukti E02: Pembuktian Strict Intra-Order FIFO Ordering

Bukti E03: Simulasi Mid-Flight Failure & Pemulihan (Recovery)

Menginjeksi kegagalan (FAIL_AFTER=2) saat memproses PAY-002 tepat sebelum commit. Saat recovery dijalankan, PAY-002 dibaca ulang dari committed offset terakhir dan seluruh 8 event tuntas diproses tanpa ada yang hilang.



Gambar: Bukti E03: Simulasi Mid-Flight Failure & Pemulihan (Recovery)

3. Checkpoint Evaluasi & Refleksi Teknis (Bab 5)

1. Apa fungsi partition pada Kafka?

Jawaban: Partisi adalah unit fundamental untuk penskalaan throughput (*horizontal scalability*) dan paralelisme throughput di Kafka. Partisi memungkinkan pemisahan penyimpanan data pada storage terpisah, sekaligus menjadi batas teritori jaminan urutan data (*strict FIFO ordering hanya dijamin per partisi*).

2. Mengapa beberapa consumer dalam consumer group yang sama menerima partition yang berbeda?

Jawaban: Karena Kafka menerapkan model *competing consumers* di mana setiap partisi dalam satu topic hanya ditugaskan (*assigned*) secara eksklusif kepada satu consumer worker di dalam grup yang sama pada satu waktu. Mekanisme ini mencegah terjadinya konsumsi ganda oleh anggota grup yang sama dan memungkinkan pemrosesan paralel yang terkoordinasi.

3. Apa fungsi committed offset?

Jawaban: Committed offset berfungsi sebagai penanda posisi baca terakhir (*checkpoint bookmark*) yang disimpan permanen oleh broker Kafka pada internal topic '__consumer_offsets'. Nilai ini memungkinkan consumer melanjutkan konsumsi data dari titik terakhir secara persis ketika terjadi restart proses, server crash, atau rebalance grup.

4. Apa yang terjadi pada partition assignment ketika salah satu consumer dalam group mengalami failure?

Jawaban: Kafka Group Coordinator mendeteksi hilangnya consumer melalui mekanisme *heartbeat timeout*, lalu segera memicu proses **Rebalance**. Partisi-partisi yang sebelumnya dipegang oleh consumer yang mati akan didistribusikan ulang kepada consumer aktif yang masih hidup sehingga alur streaming tidak macet.

5. Berdasarkan hasil eksperimen, apa perbedaan At-least-once dan Exactly-once?

Jawaban: Pada **At-least-once**, jika kegagalan terjadi setelah proses bisnis selesai namun sebelum offset berhasil di-commit, event tersebut akan dibaca dan diproses ulang saat recovery, berpotensi menimbulkan duplikasi (pada eksperimen: 13 total output vs 10 unique events). Sebaliknya, pada **Exactly-once**, produksi record dan commit offset dibungkus dalam transaksi atomik Kafka; bila terjadi crash, transaksi dibatalkan (*aborted*) sehingga consumer berisolasi *read_committed* hanya menerima output final tanpa duplikasi sama sekali (10 total output vs 10 unique events).

Refleksi Teknis (Bab 5.3)

“Keputusan arsitektur apa yang paling menentukan perilaku pipeline saat consumer mengalami failure, dan apa konsekuensinya terhadap ordering, recovery, atau duplicate processing?”

Keputusan arsitektur yang paling menentukan adalah kombinasi antara pemilihan Event Key ('order_id') dan Strategi Commit Offset manual pasca-pemrosesan record (enable.auto.commit = False):
1. Penentuan key berbasis order_id menjamin seluruh event siklus hidup order (CREATED -> PAID -> SHIPPED) selalu terpetakan secara deterministik ke partisi yang sama melalui Murmur2 hashing. Konsekuensinya, meskipun consumer mengalami crash atau terjadi dynamic rebalance grup, jaminan urutan FIFO per order tetap terjaga 100% konsisten.
2. Strategi manual commit setelah pemrosesan record menjamin semantik At-least-once: data transaksi

pembayaran tidak akan pernah hilang (zero loss) saat terjadi kegagalan sistem. Konsekuensi rekayasanya adalah kemungkinan terjadinya pemrosesan ulang (reprocessing) terhadap pesan yang belum sempat ter-commit saat crash terjadi, yang secara arsitektural dapat dimitigasi dengan penerapan logika idempoten pada consumer.